Building a GPT from First Principles

Submitted to: Saad Musema | Course: Building GPT from First Principles | November 2025

1. What I Implemented

I implemented a complete GPT-style (decoder-only) Transformer from scratch in PyTorch and trained it on the *tinyshakespeare* dataset (~1 MB of Shakespeare text). The focus was on architectural correctness and clarity of design. Every component mirrors the theory covered in the lecture.

1.1 Tokenization & Data Preparation

- Character-level tokenizer: 65-character vocabulary (a–z, A–Z, punctuation, space). No OOV tokens; the model learns spelling and formatting directly.
- Data split: 90% training / 10% validation. Validation is held-out text used to detect overfitting early.
- Chunking: block size = 128 characters. Input = chars 0–127, Target = chars 1–128 (shifted by 1 — next-token prediction).

1.2 Model Architecture

- **Token + Positional Embeddings:** Character IDs mapped to 256-dim dense vectors. A learned positional embedding encodes sequence position. Both summed: x = tok_emb + pos_emb.
- **Multi-Head Causal Self-Attention (8 heads, d_k = 32 per head):** Each token projects into Q, K, V. Score = $Q \cdot K^T$ / sqrt(d_k); softmax yields attention weights; output = weighted sum of V. A causal mask sets future positions to –inf before softmax, enforcing left-to-right generation.
- **Feed-Forward Network:** Two linear layers with GELU activation per token position. Hidden dim = 4 × 256 = 1024. FFN(x) = W2 · GELU(W1·x + b1) + b2.
- **Residual + LayerNorm:** Each sublayer wrapped with x + sublayer(LayerNorm(x)). Residuals stabilise gradient flow; Pre-LN reduces sensitivity to initialisation.
- **Decoder Stack:** 6 identical blocks stacked. Early layers capture character/word patterns; deeper layers capture grammar and style. Total parameters: ~5.7M.

1.3 Training Process

- **Loss:** Cross-entropy next-token prediction — Lt = –log P(wt | w1:t–1), averaged over all positions. Random baseline L ≈ 4.17; target L < 1.5.
- **Optimizer:** AdamW (Adam + decoupled weight decay = 0.1) — prevents overfitting, improves generalisation.
- **LR Schedule:** Linear warmup (200 steps, 0 → 3e-4), then cosine decay to 10% of peak. Avoids early divergence and stale updates.
- **Gradient Clipping:** ||g|| > 1.0 → scale down. Prevents exploding gradients in the deep stack.
- **Checkpointing:** Best model saved by lowest validation loss. Loss curves plotted for both train and val at every 500 steps.

1.4 Text Generation

The trained model generates text autoregressively — one character at a time — using temperature scaling and top-k sampling to balance creativity and coherence. It was tested both from an empty prompt and with a custom seed ("To be or not to be").

2. What I Understood from This Training

Building this model from scratch gave me a concrete understanding of why each design decision in GPT exists and what problem it solves — not just what the code does.

Why Transformers Replaced RNNs

RNNs process tokens sequentially: a 100-token sequence requires 100 serial steps, making parallelism impossible. Gradients also vanish over long sequences, causing the model to forget early context. The Transformer's self-attention attends to all tokens in a single parallel operation, making training faster and long-range dependencies directly accessible — "France" can attend to "fluent" in one step regardless of distance.

Self-Attention as Learned Information Routing

I now see self-attention as a routing mechanism: each token queries what it needs (Q), every other token advertises what it represents (K), and the dot product measures relevance. The output is a learned weighted blend of values (V). Multi-head attention runs this in parallel across 8 subspaces, allowing different heads to specialise — one tracks the previous token, another resolves coreference, another detects punctuation.

Causal Masking: The Key to Generation

Setting future attention scores to –inf before the softmax is what makes GPT generative. During training, all positions are optimised simultaneously (efficient), yet each position only sees its past (correct). At inference, the model generates one token, appends it to the context, and repeats — writing one character at a time, like a human author. Without this mask, the model would trivially cheat by reading ahead.

Depth, Residuals, and Stability

Stacking 6 decoder blocks lets the model build increasingly abstract representations. Residual connections are non-negotiable in deep networks: they create a direct gradient highway back to early layers, preventing vanishing gradients. LayerNorm stabilises activations across features, making the model robust to random initialisation. Together, these techniques are what make training a 6-layer network practical.

Next-Token Loss as a Universal Objective

Training with a single loss — predict the next character — implicitly teaches grammar, vocabulary, dialogue structure, and poetic meter without any hand-crafted labels. This simplicity is why GPT scales: the same objective applies to Shakespeare, code, or conversation. Perplexity (PPL = e^{L}) gave me an intuitive signal — a well-trained model's perplexity drops far below the random baseline of ~65.

Training Dynamics and Diagnosis

Tracking both train and val loss simultaneously revealed the model's learning state. Both losses high = underfitting (increase capacity or data). Train drops while val rises = overfitting (activate dropout, early stop). Both low with a small gap = the ideal regime. AdamW weight decay, dropout, and checkpoint selection all serve to maintain this balance — each is a concrete solution to a concrete failure mode I can now identify.

Submitted in partial fulfillment of Building GPT from First Principles — Saad Musema, November 2025